

Unit 6 : Basic Network Management

- 6.1 IP address configuration: settings in a network. TCP/IP Network address, TCP/IP Configuration files, Network Interfaces and Routes: ifconfig, route, ping, netstat, tcpdump commands.**
 - 6.2 DHCP Configuration: Configuring DHCP Client and Server, Dynamic Address, Fixed Addresses.**
 - 6.3 NIS, NFS, SAMBA introduction.**
 - 6.4 Firewall and Internet Security: Limiting Network Services, Designing Firewall.**
 - 6.5 Network Intrusion Detection: Host based Intrusion Detection Software using tripwire any relevant utility.**
-

6.4 : Firewall and Internet Security: Limiting Network Services, Designing Firewall.

6.4 : Firewall and Internet Security:

Firewall

A firewall is a network security device that monitors incoming and outgoing network traffic and permits or blocks data packets based on a set of security rules. Its purpose is to establish a barrier between your internal network and incoming traffic from external sources (such as the internet) in order to block malicious traffic like viruses and hackers.

Firewall is a network security system that filters and controls the traffic on a predetermined set of rules. This is an intermediary system between the device and the internet.

Need to configure a firewall for our Linux machine

If you have a Linux machine, you are guaranteed a certain level of security by default, courtesy the amazing Linux developer community. Linux systems are generally immune to a majority of viruses and other threats that many other operating systems succumb to. But with the increase in the volume, variety and intensity of cyber threats today, configuring a Linux firewall is quite a necessity.

A Linux Firewall

A **Linux firewall** is a device that inspects Network traffic (Inbound /Outbound connections) and makes a decision to pass or filter out the traffic. Iptables is a CLI tool for managing firewall rules on a Linux machine.

Network Security evolved with different types of Linux firewall in the era. Traditional packet-filtering firewalls deal with Routing and filtering packets (OSI Layers 3 and 4), Where else NGFWs will work with additional functions as with OSI layers (L4-L7 of OSI model).

Most of the Linux distro's ship with default firewall tools that can be used to configure them. **"IPTables"** is the default tool provided in Linux to establish a firewall. Iptables is used to set up, maintain and inspect the tables of the IPv4 and IPv6 packet filter rules in the Linux Kernel. Various network services in Linux can also be limited/restricted using command line utility i.e. Wondershaper, firewalld, etc.

Types of Firewalls:

- **Packet filtering firewalls**

A small amount of data is analyzed and distributed according to the filter's standards.

- **Proxy service firewalls**

Network security system that protects while filtering messages at the application layer.

- **Stateful multilayer inspection (SMI) firewalls**

Dynamic packet filtering that monitors active connections to determine which network packets to allow through the Firewall.

- **Next Generation Firewall (NGFW)**

Deep packet inspection Firewall with application-level inspection.

- **Network address translation (NAT) firewalls**

Allow multiple devices with independent network addresses to connect to the internet using a single IP address, keeping individual IP addresses hidden.

6.4 A : Firewall and Internet Security: Limiting Network Services:

Limiting Network Access Using FirewallD

Firewalld is a firewall management tool for Linux operating systems. As a Linux user, you can opt either to allow or restrict network access to some services or IP addresses using the firewalld firewall which is native to CentOS/RHEL 8 and most RHEL based distributions such as Fedora. The firewalld firewall uses the firewall-cmd command-line utility to configure firewall rules.

In Linux, the firewall service, **firewalld**, is enabled by default. The service is controlled by the **systemctl** command.

Note: All the commands need sudo privileges.

Before we can perform any configurations, let's first enable the firewalld service using the systemctl utility as shown:

```
$ sudo systemctl enable firewalld
```

Once enabled, you can now start **firewalld** service by executing:

```
$ sudo systemctl start firewalld
```

You can verify the status of **firewalld** by running the command:

```
$ sudo systemctl status firewalld
```

The output below confirms that the **firewalld** service is up and running.

```
tecmin@rhel8:~  
File Edit View Search Terminal Help  
[tecmin@rhel8 ~]$ sudo systemctl status firewalld  
● firewalld.service - firewalld - dynamic firewall daemon  
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; vendor  
   Active: active (running) since Wed 2020-04-22 14:37:48 EAT; 1 day 3h ago  
     Docs: man:firewalld(1)  
  Main PID: 854 (firewalld)  
    Tasks: 2 (limit: 11525)  
  Memory: 28.3M  
   CGroup: /system.slice/firewalld.service  
           └─854 /usr/libexec/platform-python -s /usr/sbin/firewalld --nofor  
Apr 22 14:37:44 rhel8 systemd[1]: Starting firewalld - dynamic firewall daem  
Apr 22 14:37:48 rhel8 systemd[1]: Started firewalld - dynamic firewall daemo  
lines 1-12/12 (END)
```

Check Firewalld Status

Configuring Rules using Firewalld

Firewalld allows you to add and block ports, blacklist, as well as whitelist IP, addresses to provide access to the server. Once done with the configurations, always ensure that you reload the firewall for the new rules to take effect.

Adding a TCP/UDP Port

To add a port, say port **443** for **HTTPS**, use the syntax below. Note that you have to specify whether the port is a TCP or UDP port after the port number:

```
$ sudo firewall-cmd --add-port=22/tcp --permanent
```

Similarly, to add a **UDP** port, specify the **UDP** option as shown:

```
$ sudo firewall-cmd --add-port=53/udp --permanent
```

The **--permanent** flag ensures that the rules persist even after a reboot.

Blocking a TCP/UDP Port

To block a TCP port, like port **22**, run the command.

```
$ sudo firewall-cmd --remove-port=22/tcp --permanent
```

Similarly, blocking a **UDP** port will follow the same syntax:

```
$ sudo firewall-cmd --remove-port=53/udp --permanent
```

Allowing a Service

Network services are defined in the **/etc/services** file. To allow a service such as **https**, execute the command:

```
$ sudo firewall-cmd --add-service=https
```

Blocking a Service

To block a service, for instance, **FTP**, execute:

```
$ sudo firewall-cmd --remove-service=https
```

Whitelisting an IP address

To allow a single IP address across the firewall, execute the command:

```
$ sudo firewall-cmd --permanent --add-source=192.168.2.50
```

You can also allow a range of IPs or an entire subnet using a CIDR (Classless Inter-Domain Routing) notation. For example to allow an entire subnet in the **255.255.255.0** subnet, execute.

```
$ sudo firewall-cmd --permanent --add-source=192.168.2.0/24
```

Removing a Whitelisted IP address

If you wish to remove a whitelisted IP on the firewall, use the `--remove-source` flag as shown:

```
$ sudo firewall-cmd --permanent --remove-source=192.168.2.50
```

For the entire subnet, run:

```
$ sudo firewall-cmd --permanent --remove-source=192.168.2.50/24
```

Blocking an IP address

So far, we have seen how you can add and remove ports and services as well as whitelisting and removing whitelisted IPs. To block an IP address, ‘**rich rules**’ are used for this purpose.

For example to block the IP **192.168.2.50** run the command:

```
$ sudo firewall-cmd --permanent --add-rich-rule="rule family='ipv4' source address='192.168.2.50' reject"
```

To block the entire subnet, run:

```
$ sudo firewall-cmd --permanent --add-rich-rule="rule family='ipv4' source address='192.168.2.0/24' reject"
```

Saving Firewall Rules

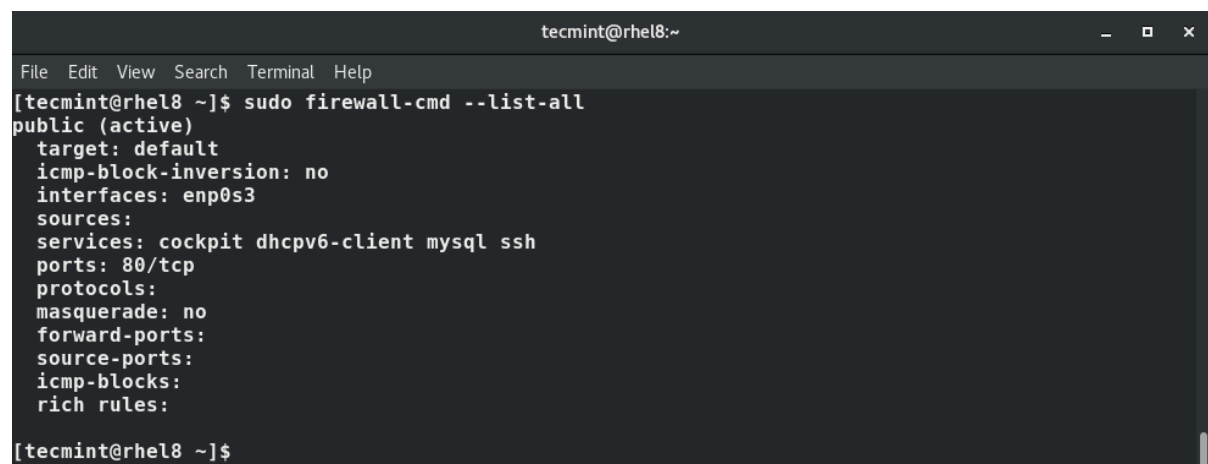
If you have made any changes to the firewall rules, you need to run the command below for the changes to be applied immediately:

```
$ sudo firewall-cmd --reload
```

Viewing the Firewall Rules

To have to peek at all the rules in the firewall, execute the command:

```
$ sudo firewall-cmd --list-all
```

A terminal window titled 'tecmintrhel8:~' with a menu bar (File, Edit, View, Search, Terminal, Help). The command '[tecmintrhel8 ~]\$ sudo firewall-cmd --list-all' has been executed. The output is as follows:

```
public (active)
target: default
icmp-block-inversion: no
interfaces: enp0s3
sources:
services: cockpit dhcpv6-client mysql ssh
ports: 80/tcp
protocols:
masquerade: no
forward-ports:
source-ports:
icmp-blocks:
rich rules:
```

The prompt '[tecmintrhel8 ~]\$' is visible at the bottom.

View Firewall Rules

6.4 B: Firewall and Internet Security: Designing Firewall.

Firewall design

Firewall design includes an organization's overall security policy decisions such as which firewall features to use, where to enforce the firewall, and, ultimately, how to configure the firewall.



The five sequential steps to follow when designing a firewall include:

1. Identify security requirements for the organization. Evaluate security requirements, assess security posture and use the information in defining the security requirements.
2. Define an overall security policy. A well-defined security policy includes network resources, access policies and authorization controls and ensures that the firewall addresses all security requirements.
3. Define a firewall philosophy. Identifying resources, applications, and services to be protected against threats from outside the organization and insider attacks makes it easier to define and configure the firewall.
4. Identify permitted communications. Define an acceptable use policy to specify the types of network activities such as applications that are allowed and denied on the LAN and the Internet Web services.
5. Identify the firewall enforcement points. Determining enforcement points is fundamental to firewall design. Firewalls are deployed at the edge, between the private LAN and a public network, such as the Internet.

Working of Linux Firewall :

Most of the Linux distro's ship with default firewall tools that can be used to configure them. We will be using "IPTables" the default tool provided in Linux to establish a firewall. Iptables is used to set up, maintain and inspect the tables of the IPv4 and IPv6 packet filter rules in the Linux Kernel.

Note:- All the command below need sudo privileges.

Chains :-

Chains are a set of rules defined for a particular task.

We have three chains(set of rules) which are used to process the traffic:-

INPUT Chains

OUTPUT Chains

FORWARD Chains

1. INPUT Chains

Any traffic coming from the internet(network) towards your local machine has to go through the input chains. That means they have to go through all the rules that have been set up in the Input chain.

2. OUTPUT Chains

Any traffic going from your local machine to the internet needs to go through the output chains.

3. FORWARD Chain

Any traffic which is coming from the external network and going to another network needs to go through the forward chain. It is used when two or more computers are connected and we want to send data between them.

Different Policies :-

There are three actions which the iptables can perform on the traffic

ACCEPT

DROP

REJECT

1. ACCEPT

When traffic passes the rules in its specified chain, then the iptable accepts the traffic. That means it opens up the gate and allows the person to go inside the kingdom of Thanos.

2. DROP

When the traffic is unable to pass the rules in its specified chain, the iptable blocks that traffic.

That means the firewall is closed.

3. REJECT

This type of action is similar to the drop action but it sends a message to the sender of the traffic stating that the data transfer has failed.

As a general rule, use REJECT when you want the other end to know the port is unreachable' use DROP for connections to hosts you don't want people to see.

NOTE:-

You need to keep in mind a simple rule here:-

The Rules you set in the iptables are checked from the topmost rules to the bottom.

Whenever a packet passes any of the top rules, it is allowed to pass the firewall. The lower rules are not checked. So be careful while setting up rules.

Basic iptables commands :

1. List the current rules of iptable :

To list the rules of the current iptables:-

sudo iptables -L

The Output would be:-



```
theprophet ~ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
theprophet ~ GeeksforGeeks
```

As you can see, we have three chains (INPUT, FORWARD, OUTPUT). We can also see column headers, but they are no actual rules. This is because most of the Linux come with no predefined rules.

Let see what each column mean.

Target:-

This defines what action needs to be done on the packet (ACCEPT,DROP,etc..)

Prot:-

This defines the protocol (TCP,IP) of the packet.

Source:-

This tells the source address of the packet.

Destination:-

This defines the destination address of the packet

2. Clear the rules :

If you ever want to clear/flush out all the existing rules. Run the following command:-
`sudo iptables -F`
This will reset the iptables.

3. Changing the default policy of chains :

`sudo iptables -P Chain_name Action_to_be_taken`

As you can see in the above picture, the default policy of each of the chain is ACCEPT.

For eg:-

If you see the forward chain, you will see “Chain FORWARD (policy ACCEPT)”. This means your computer allows any traffic to be forwarded to another computer.

In order to change the policy of forwarding to drop:-

`sudo iptables -P FORWARD DROP`

The above command will stop any traffic to be forwarded through your system. That means no other system can use your system as an intermediary to pass the data.

Making your First Rule :

1. Implementing a DROP rule :

We'll now start building our firewall policies. We'll first work on the input chain since that is where the incoming traffic will be sent through.

Syntax:-

`sudo iptables -A/-I chain_name -s source_ip -j action_to_take`

We'll take an example to understand the topic.

Let's assume we want to block the traffic coming from an IP address 192.168.1.3. The following command can be used:-

`sudo iptables -A INPUT -s 192.168.1.3 -j DROP`

This may look complicated, but most of it will make sense when we go over the components:-

-A INPUT :-

The flag -A is used to append a rule to the end of a chain. This part of the command tells the iptable that we want to add a rule to the end of the INPUT chain.

-I INPUT:-

In this flag the rules are added to the top of the chain.

-s 192.168.1.3:-

The flag -s is used to specify the source of the packet. This tells the iptable to look for the packets coming from the source 192.168.1.3

-j DROP

This specifies what the iptable should do with the packet.

In short, the above command adds a rule to the INPUT chain which says, if any packet arrives whose source address is 192.168.1.3 then drop that packet, that means do not allow the packet reach the computer.

Once you execute the above command you can see the changes by using the command:-

sudo iptables -L

The Output would be:-

```
theprophet ~$ sudo iptables -A INPUT -s 192.168.1.3 -j DROP
theprophet ~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP      all  --  192.168.1.3            anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
theprophet ~$ GeeksforGeeks
```

2. Implementing a ACCEPT rule :

If you want to add rules to specific ports of your network, then the following commands can be used.

Syntax:-

sudo iptables -A/-I chain_name -s source_ip -p protocol_name --dport port_number -j Action_to_take

-p protocol_name:-

This option is used to match the packets that follow the protocol protocol_name.

-dport port_number:

This option is available only if you give the -p protocol_name option. It specifies to look for the packets that are going to the port "port_number".

Example:-

Let's say we want to keep our SSH port open (we will assume in this guide that the default SSH port is 22) from the 192.168.1.3 network we blocked in the above case. That is we only want to allow those packets coming from 192.168.1.3 and which wants to go to the port 22.

What do we do:-

Let's try the below command:-

sudo iptables -A INPUT -s 192.168.1.3 -p tcp --dport 22 -j ACCEPT

The above command says looks for the packets originating from the IP address 192.168.1.3, having a TCP protocol and who wants to deliver something at the port 22 of my computer. If you find those packets then Accept them.

The Output for the command is:-

```
theprophet ~$ sudo iptables -A INPUT -s 192.168.1.3 -p tcp --dport 22 -j ACCEPT
theprophet ~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP       all  --  192.168.1.3            anywhere
ACCEPT     tcp  --  192.168.1.3            anywhere    tcp dpt:ssh

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
theprophet ~$ GeeksforGeek
```

The Rules you set in the iptables are checked from the top to the bottom. Whenever a packet is processed to one of the top rules, it is not checked with the lower rules.

In our case, The packet was checked with the topmost rule, which says that the iptable must drop any packet coming from 192.168.1.3. Hence once the packet got accessed through this rule, it did not go to the next rule which allowed packets to the port 22. Therefore it failed.

What could be done?

The easiest answer is, Add the rule to the top of the chain. All you need to do is change the -A option to -I option. (In our scenario we first delete the rule [refer the next section] added in the above section and then add the below rule again)

The command to do that is:-

sudo iptables -I INPUT -s 192.168.1.3 -p tcp --dport 22 -j ACCEPT

Now check the iptable configuration using -L command. The output would be:-

```

theprophet ~ sudo iptables -I INPUT -s 192.168.1.3 -p tcp --dport 22 -j ACCEPT
theprophet ~ sudo iptables -L
Chain INPUT (policy ACCEPT)
target    prot opt source                destination           tcp dpt:ssh
ACCEPT    tcp  --  192.168.1.3            anywhere
DROP      all  --  192.168.1.3            anywhere

Chain FORWARD (policy ACCEPT)
target    prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target    prot opt source                destination
theprophet ~ GeeksforGeek

```

Therefore, any packet coming from 192.168.1.3 is first checked if it is going to the port 22 if it isn't then it is run through the next rule in the chain. Else it is allowed to pass the firewall.

Now that you have understood how to block and accept the incoming traffic let's see how to delete rules:-

3. Deleting a rule from the iptable :

Syntax:-

sudo iptables -D chain_name rule_number

Example:-

If we want to delete the rule which accepts the traffic to port 22 and which we have added in the previous section, then:-

sudo iptables -D INPUT 1

Remember the rule number starts from 1

The Output :-

```

theprophet ~ sudo iptables -D INPUT 1
theprophet ~ sudo iptables -L
Chain INPUT (policy ACCEPT)
target    prot opt source                destination
DROP      all  --  192.168.1.3            anywhere

Chain FORWARD (policy ACCEPT)
target    prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target    prot opt source                destination
theprophet ~ GeeksforGeeks

```

4. Saving your configuration :

This part is unnecessary if you are implementing it on a personal computer which is not a server, but if

you are implementing a firewall on a server, then there are high chances that your server might get corrupted and you might lose all your data. So, it's always better to save your configurations.

There are a lot of ways to do this, but the easiest way I find is with an iptables-persistent package. You can download the package from Ubuntu's default repositories:

```
sudo apt-get update  
sudo apt-get install iptables-persistent
```

Once the installation is complete, you can save your configuration using the command:-

```
sudo invoke-rc.d iptables-persistent save
```

Commands Summary :

1. List the current rules of iptables:

```
sudo iptables -L
```

2. To change the default policy:

```
sudo iptables -P Chain_name Action_to_be_taken
```

Example:-

```
sudo iptables -P FORWARD DROP
```

3. To clear/flush all the rules

```
sudo iptables -F
```

4. To append a rule at the end of the chain:

```
sudo iptables -A
```

5. To append a rule at the start of the chain:

```
sudo iptables -I
```

6. To implement a ACCEPT rule:-

```
sudo iptables -A/-I chain_name -s source_ip -j action_to_take
```

Example:-

```
iptables -A INPUT -s 192.168.1.3 -j ACCEPT
```

7. To implement a DROP rule:-

```
sudo iptables -A/-I chain_name -s source_ip -j action_to_take
```

Example:-

```
iptables -A INPUT -s 192.168.1.3 -j DROP
```

8. Implementing rules on specific ports/protocols:-

```
sudo iptables -A/-I chain_name -s source_ip -p protocol_name --dport  
port_number -j Action_to_take
```

Example:-

```
sudo iptables -I INPUT -s 192.168.1.3 -p tcp --dport 22 -j ACCEPT
```

9. To delete a rule:-

```
sudo iptables -D chain_name rule_number
```

Example:-

```
sudo iptables -D INPUT 1
```

10. To save the configuration:-

```
sudo invoke-rc.d iptables-persistent save
```